

Projet — Analyse de la Polarisation

Point d'avancement technique

17 mars 2026

1 Objet du document

Ce document synthétise l'état d'avancement du projet *Analyse de la Polarisation* à ce stade. Il ne s'agit pas encore du rapport final demandé dans le sujet, mais d'un compte rendu de travail déjà effectué dans le dépôt.

2 Rappel du cadre du projet

Le sujet demande de travailler sur deux types de profils électoraux :

- les votes par approbation, représentés par des vecteurs binaires ;
- les votes par ordres totaux, représentés par des permutations des candidates.

Le projet doit ensuite :

- générer des profils avec un niveau de polarisation contrôlable ;
- implémenter plusieurs mesures de polarisation ;
- implémenter des distances entre bulletins ;
- proposer une estimation de la polarisation via un schéma de type consensus global / consensus à deux groupes ;
- produire des expériences et des graphiques ;
- fournir un rapport final documenté.

3 Travail déjà réalisé

3.1 Structure du projet

Une première structure de code a été mise en place pour séparer les différentes parties du projet. Les fichiers suivants ont été créés :

- `src/types.py`
- `src/generation.py`
- `src/distances.py`
- `src/pairwise.py`
- `src/phi2.py`
- `src/consensus.py`
- `src/clustering.py`
- `src/measures.py`
- `src/experiments.py`

- `src/plots.py`
- `scripts/run_phi2_experiments.py`
- `scripts/run_distance_experiments.py`
- `main.py`
- `README.md`

L'objectif de cette organisation est de faciliter la suite du projet : chaque partie du sujet peut être reliée à un module précis.

3.2 Génération de profils

Une première version des générateurs aléatoires a été implémentée dans `src/generation.py`.

Votes par approbation La fonction `generate_approval_profile` :

- génère un bulletin central binaire ;
- construit un bulletin opposé bit à bit ;
- contrôle le niveau de polarisation à l'aide d'un paramètre `alpha` ;
- ajoute du bruit avec un paramètre `noise`.

L'idée actuelle est la suivante :

- si `alpha` est faible, la majorité des votantes restent proches d'un centre commun ;
- si `alpha` est forte, une part importante du profil est répartie entre un centre et son opposé.

Votes par ordres totaux La fonction `generate_ranking_profile` :

- génère un ordre central aléatoire ;
- utilise l'ordre inverse comme pôle opposé ;
- contrôle le niveau de polarisation avec `alpha` ;
- ajoute du bruit par une suite d'échanges aléatoires.

Cette partie permet déjà de préparer les expériences demandées aux questions 1, 2, 6 et 15.

3.3 Distances

Une première implémentation des distances demandées a été ajoutée dans `src/distances.py`.

Distance de Hamming La fonction `hamming_distance(a, b)` calcule le nombre de composantes différentes entre deux bulletins d'approbation de même taille.

Distance de Spearman Deux fonctions ont été introduites :

- `ranking_to_positions(order)` convertit une permutation en tableau de positions ;
- `spearman_distance(order1, order2)` calcule la somme des écarts absolus de rang.

Ces fonctions constituent la base de travail pour les questions 7 et 8.

3.4 Première implémentation de ϕ_2

Une première version de la mesure ϕ_2 a été codée dans `src/phi2.py`, en s'appuyant sur `src/pairwise.py`.

Remarque importante Cette partie est encore considérée comme une **première approximation de travail**. Elle permet d’avoir une chaîne exécutable pour les premiers tests et graphiques, mais la définition exacte de $d^{c_k, c_l}(p)$ devra encore être vérifiée ligne à ligne par rapport au PDF avant de figer la version du rapport final.

Autrement dit :

- le squelette algorithmique est en place ;
- l’interprétation mathématique fine de cette partie doit encore être stabilisée.

3.5 Consensus global u_1

Une première version du calcul du consensus global a été ajoutée dans `src/consensus.py`.

Cas approbation La fonction `approval_consensus_ballot` choisit pour chaque candidate la valeur majoritaire dans le profil. La quantité `u1_approval(profile)` somme ensuite les distances de Hamming entre ce consensus et tous les bulletins du profil.

Cas ordres totaux Une première approximation de consensus est construite par rang moyen :

- on calcule pour chaque candidate son rang moyen dans le profil ;
- on trie ensuite les candidates selon ce rang moyen.

La fonction `u1_ranking(profile)` somme alors les distances de Spearman entre ce consensus et les bulletins du profil.

Remarque importante Pour les votes par ordres totaux, cette version est une **version pragmatique initiale**. Le sujet indique qu’une approche plus rigoureuse doit s’appuyer sur un problème de couplage parfait de poids minimum. Cette partie devra donc être raffinée pour la version finale répondant strictement à la question 11.

3.6 Clustering à deux groupes et estimation de u_2

Le fichier `src/clustering.py` contient une première adaptation de l’algorithme de type k -means avec $k = 2$.

Le schéma utilisé est :

1. initialisation aléatoire de deux centroïdes ;
2. affectation de chaque bulletin au centre le plus proche ;
3. recalcul du centroïde de chaque groupe via le consensus du groupe ;
4. répétition jusqu’à stabilisation ;
5. répétition de plusieurs lancements et conservation du meilleur coût trouvé.

Deux fonctions ont été ajoutées :

- `kmeans2_approval`
- `kmeans2_ranking`

Le résultat retourné contient actuellement :

- le coût total ;
- les deux centroïdes ;
- les labels d’affectation.

3.7 Mesures basées sur les distances

Le fichier `src/measures.py` contient une première implémentation des mesures :

- $\phi_{d_H}(p)$ pour les votes par approbation ;
- $\phi_{d_S}(p)$ pour les votes par ordres totaux.

Les formules codées suivent l'idée :

$$\phi_{d_H}(p) = \frac{2}{nm}(u_1(p) - \widetilde{u}_2(p))$$

et

$$\phi_{d_S}(p) = \frac{4}{nm^2}(u_1(p) - \widetilde{u}_2(p)).$$

Ici, $\widetilde{u}_2(p)$ est l'estimation obtenue par la procédure itérative de clustering.

3.8 Scripts d'expériences

Deux scripts ont été ajoutés pour automatiser les premiers essais :

- `scripts/run_phi2_experiments.py`
- `scripts/run_distance_experiments.py`

Ils permettent :

- de faire varier le paramètre `alpha` ;
- de répéter plusieurs tirages aléatoires ;
- de calculer moyenne et écart-type ;
- de sauvegarder des fichiers CSV ;
- de produire des graphiques.

3.9 Visualisation

Le fichier `src/plots.py` contient une fonction `plot_curve` qui trace une courbe avec barres d'erreur à l'aide de `matplotlib`. Cela prépare directement les figures des questions 6 et 15.

3.10 Point d'entrée rapide

Le fichier `main.py` fournit un test d'exécution minimal :

- génération d'un profil approval ;
- génération d'un profil ranking ;
- calcul de ϕ_2 , ϕ_{d_H} et ϕ_{d_S} ;
- affichage de valeurs numériques.

Ce point d'entrée a été exécuté avec succès dans l'environnement courant.

4 Tests et vérifications effectuées

4.1 Installation des dépendances

Le fichier `requirements.txt` a été ajouté. Le paquet `pytest` a ensuite été installé dans l'environnement courant afin de pouvoir lancer les tests automatiques.

4.2 Jeu minimal de tests

Les fichiers suivants ont été ajoutés dans `tests/` :

- `test_generation.py`
- `test_distances.py`
- `test_phi2.py`
- `test_consensus.py`

Ces tests vérifient actuellement :

- la forme des profils générés ;
- un cas simple pour la distance de Hamming ;
- l'annulation de la distance de Spearman sur deux ordres identiques ;
- l'annulation de ϕ_2 sur un profil parfaitement homogène ;
- la cohérence de base entre u_1 et l'estimation à deux groupes.

4.3 Résultat de la campagne de tests

La commande suivante a été exécutée :

```
python -m pytest
```

Résultat observé :

- 7 tests collectés ;
- 7 tests passés.

5 Etat actuel : ce qui est solide et ce qui reste à fiabiliser

5.1 Parties déjà exploitables

Les éléments suivants sont déjà exploitables pour continuer le projet :

- la structure du dépôt ;
- les générateurs de profils ;
- les distances de base ;
- une première exécution de bout en bout ;
- des scripts d'expériences ;
- un socle de tests.

5.2 Parties encore provisoires

Les points suivants devront être améliorés avant la rédaction finale :

- la définition exacte de $d^{c_k, c_l}(p)$ et donc de ϕ_2 ;
- l'alignement strict de toutes les formules avec le PDF ;
- la partie théorique des preuves pour les questions 4, 7, 9, 10 et 11 ;
- le calcul de u_1 dans le cas des ordres totaux via une formulation conforme à l'indication du sujet ;
- la production des graphes finaux avec paramétrage propre et commentaire rédactionnel.

6 Prochaines étapes recommandées

L'ordre de travail recommandé pour la suite est le suivant :

1. corriger et stabiliser la définition mathématique de ϕ_2 ;
2. aligner les fonctions `pairwise.py` et `phi2.py` avec le sujet ;
3. raffiner le calcul de u_1 pour les ordres totaux ;
4. lancer les scripts d'expériences pour produire les premiers CSV et graphiques ;
5. commencer la rédaction du vrai rapport final question par question.

7 Conclusion

Le projet dispose maintenant d'une base de code propre, exécutable et testée. Cette base ne constitue pas encore la version finale mathématiquement stabilisée, mais elle permet de travailler efficacement sur la suite : mise au propre des définitions, expériences, graphiques et rédaction du rapport demandé.